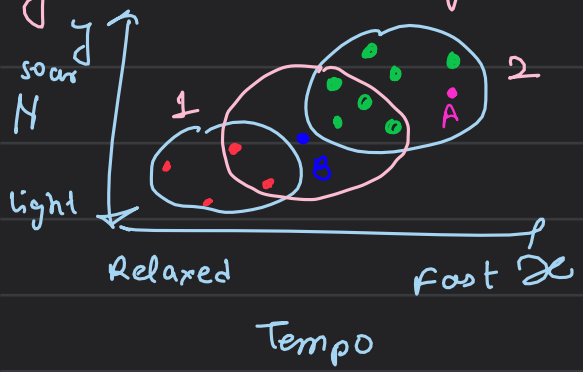
Perception-ML

What is machine learning?

↳ training of machine (model) based on past fed data so that it can judge when the new data comes

eg: K-nearest neighbours:



the blue circle consists of pre trained data, now suppose a song of medium-Intensity & medium Tempo comes

↳ where will it lie

the newly fed data will come under circle 2 based on no. of neighbours associated with B.

More data > better model > higher accuracy.

Types of Machine Learning
- Supervised Learning
- Unsupervised Learning
- Reinforced Learning

Supervised Learning: → the target variable is known

→ the output is known to you what the model does is that it just maps the output with the given set of labels

we feed the features of the data along with the label

uses labelled data to train the model

↳ output is already predefined, what the model needs to do is just to map the inputs to the outputs

* takes the labeled input & maps it to the output

Training: needs external supervision to train model

types of problems that can be solved: Classification & Regression
↳ it learns which feature is associated with which label

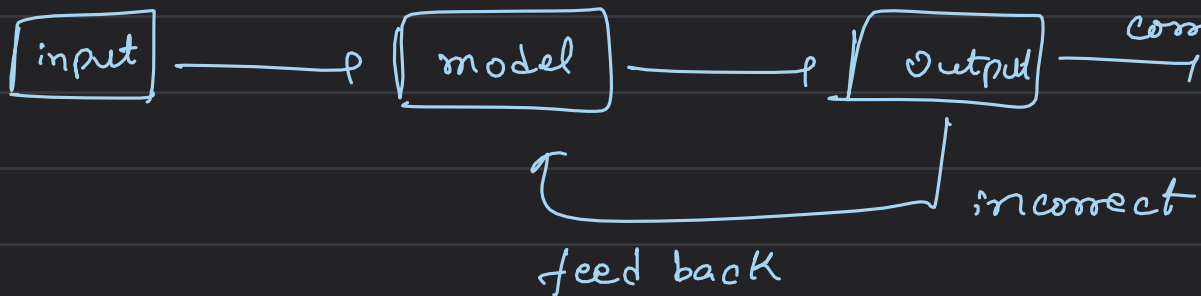
Unsupervised Learning:
does not use any labeled data to train the model
↳ there is no fixed data variable, the model learns from the data identifies patterns in it & predict the output

Understands patterns and trends in the data and discovers the output (there is no fixed output that is there not a specific set which the model will output

Training: learn on there own & train the models

types of problems: Clustering & Association that can be solved

Reinforcement learning: ↳ follows trail & error
works on the principle of -ve feed back, let say you fed an image of the dog, the model identifies it as a cat a -ve feed back will be given to the dog so next time if an image of a dog is fed, the model identifies it correctly

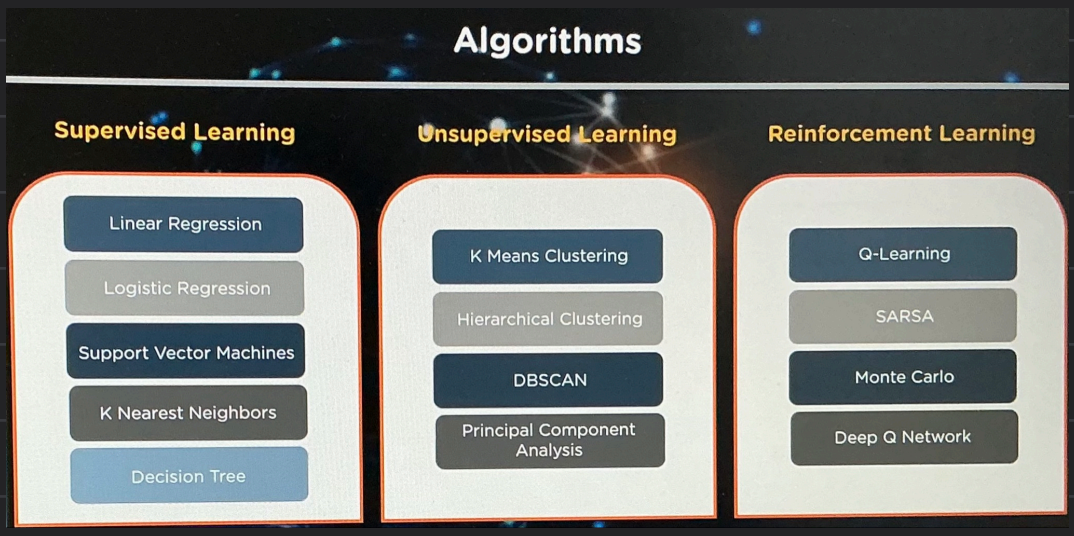


Training: no supervision required.

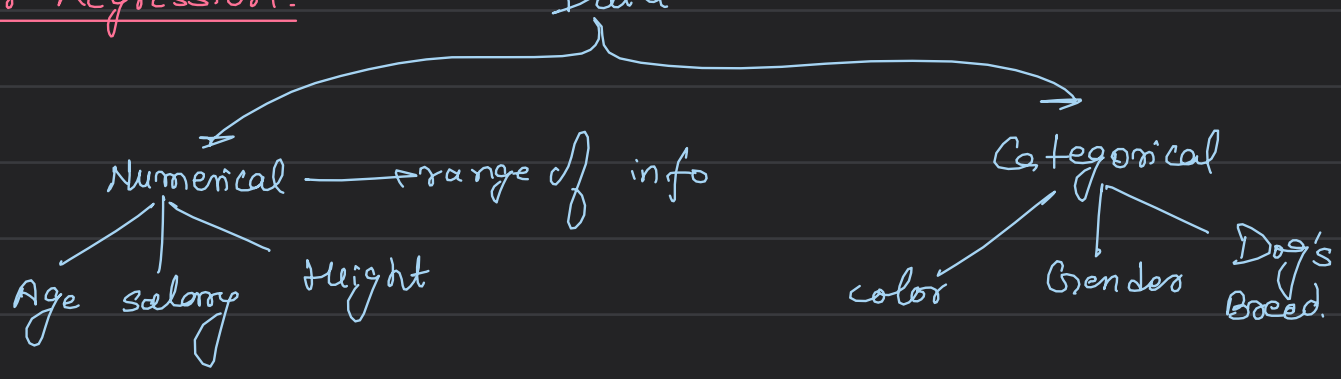
types of problems that can be solved : Reward / penalty Based

uses an agent and an environment to produce actions & rewards

⊛ there is no predefined target variable, eg: the model has to identify an input object from a given list of objects by performing trail & error eg: a square is input in the model but it identifies it as a rectangle so now a negative feedback will be sent & from the next encounter with square the model will remember it.



Linear Regression: → statistical model used to predict relation b/w independent & dependent variable



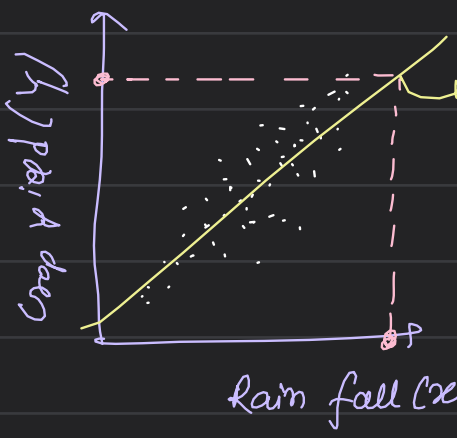
Supervised Learning

- Regression
  - Simple Linear Regression → Drawing a st line through the set of points
  - Multiple Linear Regression → multiple independent variable
  - Polynomial LR → Drawing polynomial through the set of points
- Classification (0/1)
  - Logistic Regression

Simple Linear Regression:

⊛ simplest form of LR consisting of 1 independent variable & 1 dependent variable is $y = mx + c$ consisting of (x_1, y_1) & (x_2, y_2)

eg: Rainfall vs Crop yield



the line which passes through max no. of points

↳ How to find this st. line ?

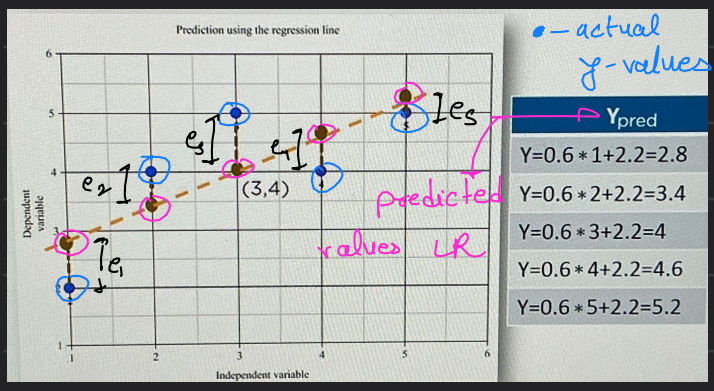
Computation: $(x, y) = \{(1, 2), (2, 4), (3, 5), (4, 4), (5, 5)\}$

↳ finding the mean

let $y = mx + c$

$$m = \frac{(n \Sigma xy - \Sigma x \Sigma y)}{(n(\Sigma x^2) - (\Sigma x)^2)}$$

$$c = \frac{\Sigma y \Sigma x^2 - \Sigma x \Sigma xy}{(n \Sigma x^2 - (\Sigma x)^2)}$$



● – actual y-values

Y_{pred}
Y=0.6*1+2.2=2.8
Y=0.6*2+2.2=3.4
Y=0.6*3+2.2=4
Y=0.6*4+2.2=4.6
Y=0.6*5+2.2=5.2

predicted values LR

The distance b/w actual & predicted values is residuals or errors

* the best fit line should have least sum of squares of these errors known as esquares

↳ minimize : $\sum_{i=1}^{n} e_i^2$

Multiple Linear Regression → multiple Linear regression

$$y = m_1x_1 + m_2x_2 + m_3x_3 + \cdots + m_nx_n + C$$

all the independent variables

Implementation :

[20]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```
→ for jupyter note book → no req. of plt.show()

[40]:
```
companies = pd.read_csv(r'C:\Users\Harshita Sinha\Downloads\Linear_regression\startup_profit_data.csv')
X = companies.iloc[:,:-1].values
y = companies.iloc[:, 4].values
companies.head()
```
→ assign all the row elements & all the column elements except the last column (i=-1)

[40]:

	RD_Spend	Administration	Marketing_Spend	State	Profit
0	165349	136898	471784	New York	192262
1	162598	151378	443000	California	191792
2	153442	101146	407935	Florida	191050
3	144373	118672	383650	New York	182902
4	142107	91392	366168	Florida	166187

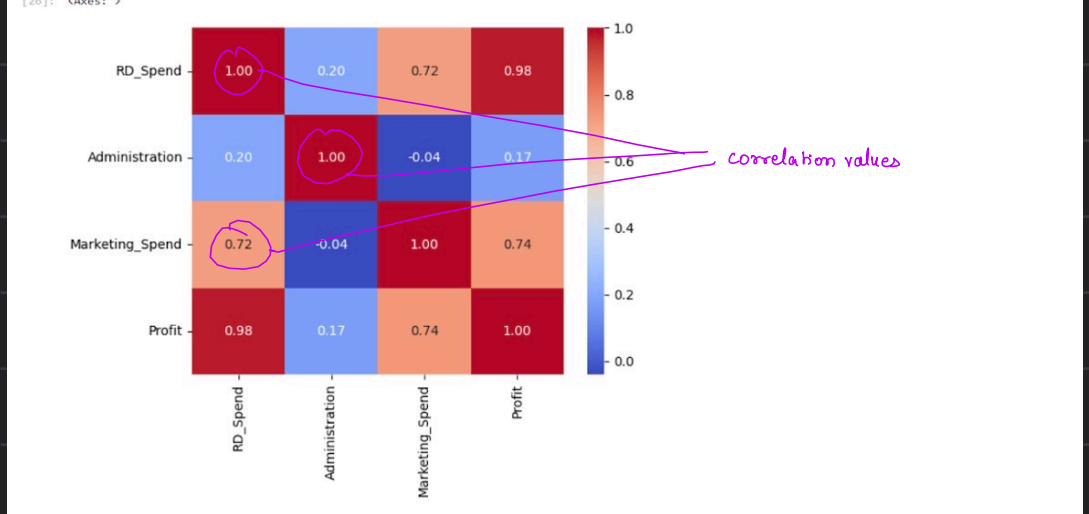
Y

X

[26]:
```
#Data visualization
#Building a correlation matrix
sns.heatmap(companies.corr(numeric_only=True), annot=True, fmt=".2f", cmap="coolwarm")
```
→ colours

→ to compute this correlation matrix

[26]: <Axes: >



correlation values

[36]:
```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

# Re-extract X fresh from the dataframe before encoding
X = companies.iloc[:, :-1].values
y = companies.iloc[:, 4].values

ct = ColumnTransformer(
    transformers=[('encoder', OneHotEncoder(drop='first'), [3])],
    remainder='passthrough'
)
X = ct.fit_transform(X)
print(X)
```

→ strings → assign numerical values

```
[[0.0 1.0 165349 136898 471784]
 [0.0 0.0 162598 151378 443000]
 [1.0 0.0 153442 101146 407935]
 [0.0 1.0 144373 118672 383650]
 [1.0 0.0 142107 91392 366168]
 [0.0 1.0 131877 99815 362861]
 [0.0 0.0 134615 147199 127717]
 [1.0 0.0 130298 145530 323877]
 [0.0 1.0 120543 148719 311613]
 [0.0 0.0 123995 128702 304982]
 [1.0 0.0 101913 110594 229161]
 [0.0 0.0 100672 91791 249745]
 [1.0 0.0 93864 127320 249840]]
```

[37]:
```
X = X[:, 1:]
```

↳ all rows & remove the first column

[39]:
```
#splitting the data into training and testing set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
```

↑ 20% testing data , 80% training data

[43]:
```
#Fitting multiple linear regression to the training set
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()#multiple linear regression happeningb in the back
regressor.fit(X_train, y_train)
```

↳ computing $y = m_1x_1 + m_2x_2 + m_3x_3 + m_4x_4 + c$ ← the curve of linear Regression

[43]:

LinearRegression

▾ Parameters

fit_intercept	True
copy_X	True
tol	1e-06
n_jobs	None
positive	False

Since now we have the training data set we can compute (m_1, m_2, m_3, m_4, c) using the Least sq. error method

[47]:
```
y_pred = regressor.predict(X_test)#we used 0.2 fraction of the data set for raining and now the remaining data is for testing
print(y_pred)
```

↳ Now we have to trained model So we can now test it using x_{test} & compute y_{pred}

```
[103972.48225928 130009.21427947 131562.37248257  68688.01515859
 177474.26731093 114379.66643277  64169.28696034  97350.53006151
 111990.43893984 166586.07568637]
```

[48]:
```
#Calculating the coeffs
print(regressor.coef_)
```

```
[2.17496783e+03 8.03262090e-01 6.07847293e-02 2.84990922e-02]
```

m_1 m_2 m_3 m_4

[49]:
```
#Calculating the intercepts
print(regressor.intercept_)
```

c

```
36446.21628350814
```

[50]:
```
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)
```

↳ to test how good / bad the model is

$$R^2 = 1 - \frac{\sum (y - \bar{y})^2}{\sum (y - \hat{y})^2}$$

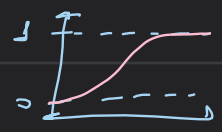
y : actual values (y_{test})

$\hat{y}$: y_{pred}

$\bar{y}$: mean of all actual values

[50]:
```
0.935584383728793
```

Logistic Regression (yes/No) → (b/w 0/1)

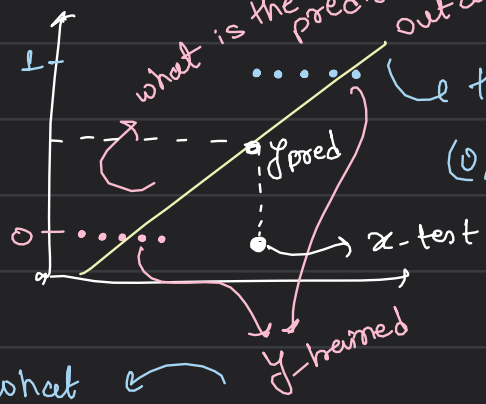


a dataset with one or more independent variables is used to determine binary o/p of the dependent variable

↓

the dependent variable outcome is discrete

Teaching the model with the given dataset → Dropping the non-essential components of the dataset → Determing the survival of passengers & evaluating the model



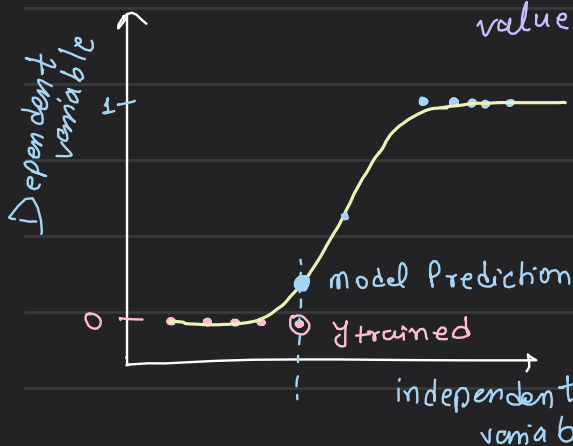
what the outcome should be

this graph formed from linear regression goes beyond (0,1) which dosen't really matter since it's a probabilit

↓

So we squeeze this graph to fit b/w (0,1) & to minimize the error $(y_{pred} - y_{test})$

we calculate likelihood in case of logistic Reg. & the curve with max value is chosen



$\log(p) + \log(1-p)$

↓

1-p

p

student Passed x logp

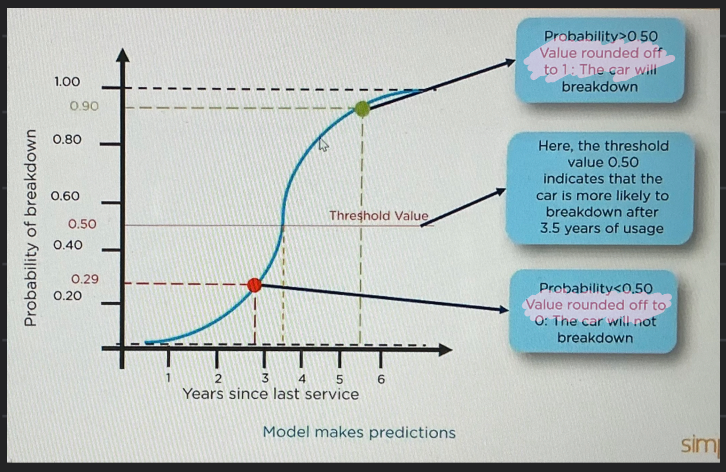
+ student failed x log(1-p)

a better way

So: $e = \log y_{pred} - \log y_{train}$

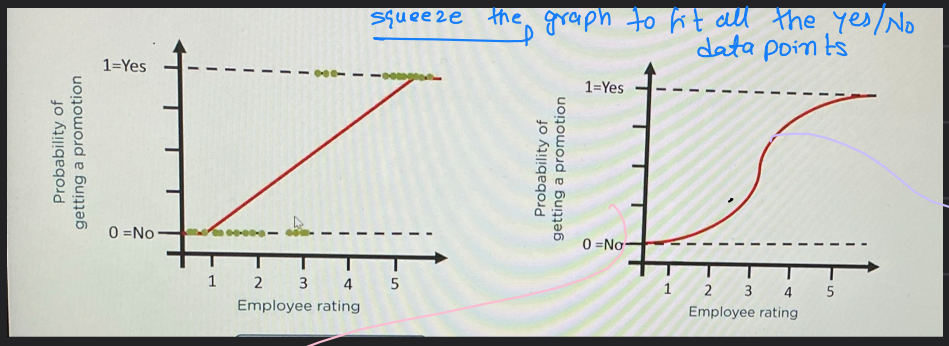
↓

not $y_{pred} - y_{train}$



* In logistic regression $y_{pred} \in (0,1)$

* In linear regression $y_{pred} \in R$



we try to fit a graph b/w the discrete points

↓

How to compute this best fit line ?

$$odds(\theta) = \frac{P(E)}{P(\bar{E})} \in (0, \infty)$$

(1) Take eqn as st line : $y = \beta_0 + \beta_1 x$

β_1 : prediction

β_0 : variable

$$\log\left(\frac{p(x)}{1-p(x)}\right) = \beta_0 + \beta_1 x \longrightarrow \frac{p(x)}{1-p(x)} = e^{\beta_0 + \beta_1 x}$$

$$p(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

sigmoid fxn $\rightarrow \frac{1}{1+e^{-x}}$

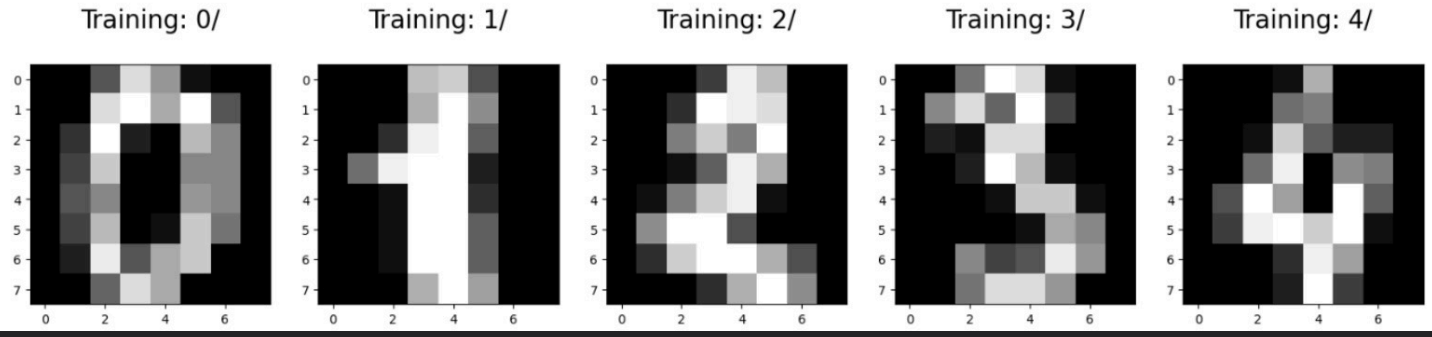
$\beta_1, \beta_0 = ?$ $y = \beta_0 + \beta_1 x$

```
[1]: from sklearn.datasets import load_digits
     from sklearn.model_selection import train_test_split
     import numpy as np
     import matplotlib.pyplot as plt
     import seaborn as sns
     from sklearn import metrics
     %matplotlib inline
```

```
[2]: digits = load_digits()
     print("Image Data Shape", digits.data.shape)
     print("Label Data Shape", digits.target.shape)
```

```
Image Data Shape (1797, 64)
Label Data Shape (1797,)
```

```
[3]: plt.figure(figsize=(20, 4))
     for index, (image, label) in enumerate(zip(digits.data[0:5], digits.target[0:5])):
         plt.subplot(1, 5, index+1)
         plt.imshow(np.reshape(image, (8, 8)), cmap = plt.cm.gray)
         plt.title('Training: %i/\n'%label, fontsize = 20)
```




```
[5]: x_train, x_test, y_train, y_test = train_test_split(digits.data, digits.target, test_size = 0.23, random_state =2)
```

23% testing dataset & 77% training dataset

```
[8]: from sklearn.linear_model import LogisticRegression
     from sklearn.preprocessing import StandardScaler

     sc = StandardScaler()
     x_train = sc.fit_transform(x_train)
     x_test = sc.transform(x_test)  # use transform only, NOT fit_transform

     logisticRegr = LogisticRegression(max_iter=1000)
     logisticRegr.fit(x_train, y_train)
```

learns the mean(0)/std(1) from training data, then applies it

applies the same mean/std to test data

Here we have '10' independent variables (0-9)

$$y = \frac{1}{1+e^{-(\beta_0+\beta_1 x_1+\beta_2 x_2+\cdots-\beta_{10}x_{10})}}$$

Now we have the points to train the model so find $\beta_0, \beta_1 \cdots \beta_{10}$

```
[8]: ▾ LogisticRegression
     ▸ Parameters
```

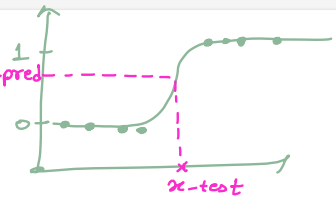
Model trained successfully

```
]: y_pred = logisticRegr.predict(x_test)
   print(y_pred)
```

using the known curve now compute y-pred based on x-test

```
[4 0 9 1 8 7 1 5 1 6 6 7 6 1 5 5 8 6 2 7 4 6 4 1 5 2 9 5 4 6 5 6 3 4 0 9 9
 8 4 6 8 8 5 7 9 8 9 6 1 7 0 1 9 7 3 3 1 8 8 8 9 8 5 8 7 8 3 5 8 4 3 9 3 8
 7 3 3 0 8 7 2 8 5 3 8 7 6 4 6 2 2 0 1 1 5 3 5 7 1 8 2 2 6 4 6 7 3 7 3 9 4
 7 0 3 5 8 5 0 3 9 2 7 3 2 0 8 1 9 2 1 5 1 0 3 4 3 0 8 3 2 2 7 3 1 6 7 2 8
 3 1 1 6 4 8 2 1 8 4 1 3 1 1 9 5 4 8 7 4 8 9 5 7 6 9 0 0 4 0 0 9 0 6 5 8 8
 3 7 9 2 0 3 2 7 3 0 2 1 5 2 7 0 6 9 3 1 1 3 5 2 3 5 2 1 2 9 4 6 5 5 5 9 7
 1 5 7 6 3 7 1 7 5 1 7 2 7 5 5 4 8 6 6 2 8 7 3 7 8 0 3 5 7 4 3 4 1 0 3 3 5
 4 1 3 1 2 5 1 4 0 3 1 5 5 7 4 0 1 0 9 5 5 5 4 0 1 8 6 2 1 1 1 7 9 6 7 9 7
 0 4 9 6 9 2 7 2 1 0 8 2 8 6 5 7 8 4 5 7 8 6 4 2 6 9 3 0 0 8 0 6 6 7 1 4 5
 6 9 7 2 8 5 1 2 4 6 8 8 7 6 0 8 0 6 1 5 7 8 0 4 1 4 5 9 2 2 3 9 1 2 9 3 2
 8 0 6 5 6 2 5 2 3 2 6 1 0 7 6 0 6 2 7 0 3 2 4 2 3 6 9 7 7 0 3 5 4 1 2 2 1
 2 7 7 0 4 9 8]
```

y-pred



[12]:
```
from sklearn.metrics import confusion_matrix

y_pred = logisticRegr.predict(x_test)
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

Creating the confusion matrix

```
[[38  0  0  0  0  0  0  0  0  0]
 [ 0 45  0  0  0  0  1  0  1  1]
 [ 0  0 43  0  0  0  0  0  0  0]
 [ 0  0  1 40  0  0  0  1  0  0]
 [ 0  0  0  0 33  0  0  1  3  1]
 [ 0  1  0  0  1 44  0  0  0  0]
 [ 0  1  0  0  0  0 39  0  1  0]
 [ 0  0  0  0  0  0  0 45  1  0]
 [ 0  0  0  0  0  0  0  0 37  1]
 [ 0  0  0  2  0  1  0  1  1 29]]
```

Rows = Actual digit (0-9)
Columns = Predicted digit (0-9)
Diagonal = Correct predictions
Off-Diagonals = Misclassification

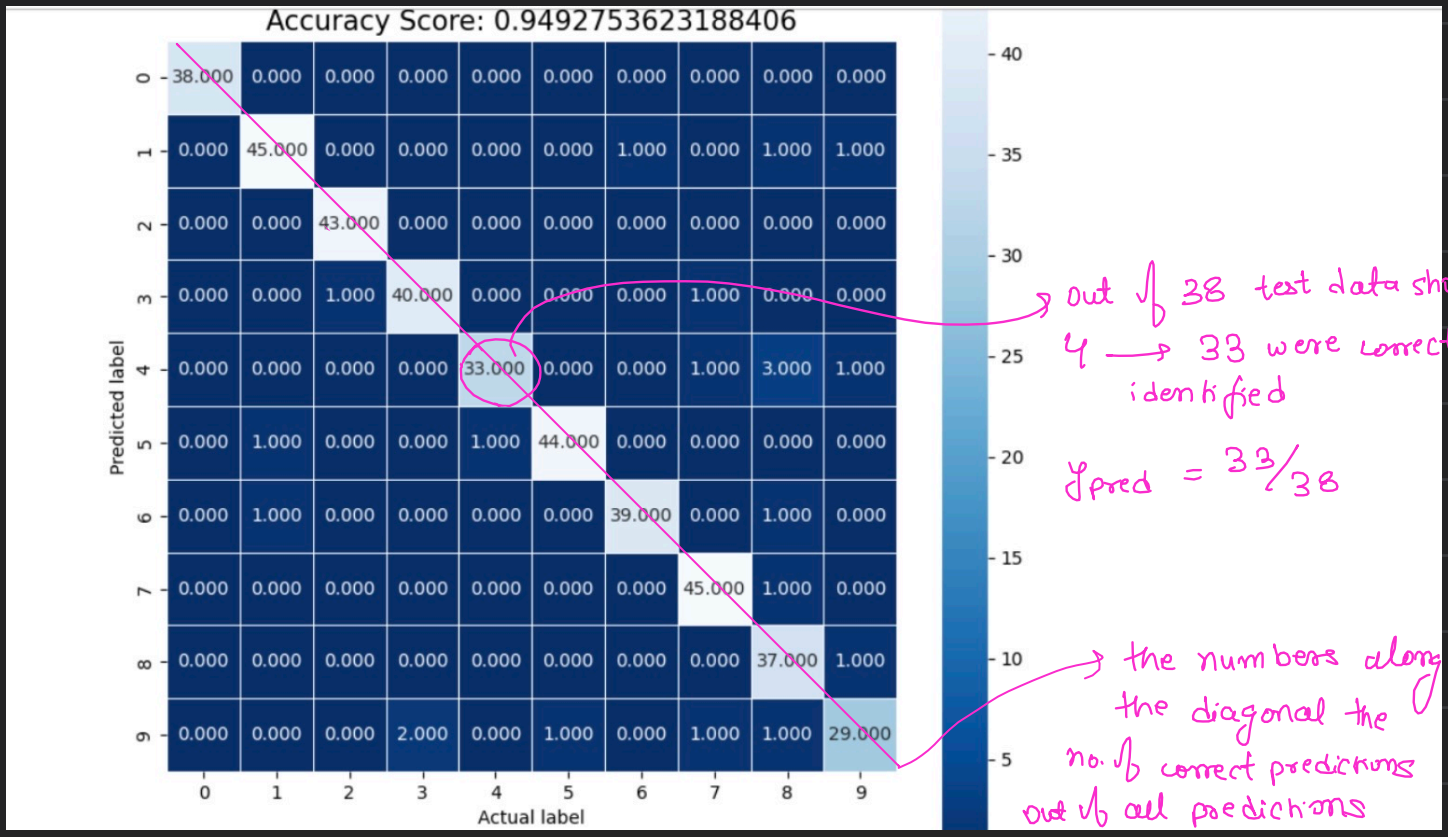
[10]:
```
score = logisticRegr.score(x_test, y_test)
print(score)
```

score = Correct Predictions / Total No. of Predictions

```
0.9492753623188406
```

[13]:
```
plt.figure(figsize=(9,9))
sns.heatmap(cm, annot = True, fmt = ".3f", linewidth = .5, square=True, cmap = 'Blues_r')
plt.xlabel("Actual label")
plt.ylabel("Predicted label")
all_sample_title = 'Accuracy Score: {0}'.format(score)
plt.title(all_sample_title, size= 15)
```

[13]:
```
Text(0.5, 1.0, 'Accuracy Score: 0.9492753623188406')
```



out of 38 test data showing 4 → 33 were correctly identified

$y_{pred} = 33/38$

the numbers along the diagonal the no. of correct predictions out of all predictions

So,

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression
X_train = [ - - - - ]
Y_train = [ - - - - ]
model.fit(X_train, Y_train)
X_test = [ - - - - - ]
Y_test = [ - - - - - ]
model.predict(X_test)   → returns y_pred
score = (y_test - y_pred)^2
```

Linear Regression	Logistic Regression
⇒ supervised regression model	⇒ supervised classification model
⇒ the dependent variable is cont^{S}	⇒ the dependent variable is discrete (0, 1) ⟶ $y_{pred} \in (0, 0.5)$ ↓ 0 ; $y_{pred} \in (0.5, 1) \rightarrow 1$
⇒ $y = mx + c$	⇒ $y = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$
⇒ we predict a new value ($\in R$) based on previous dataset	⇒ we predict True / False based on trained data set